

FamPlan: Simple Technology Stack Recommendation

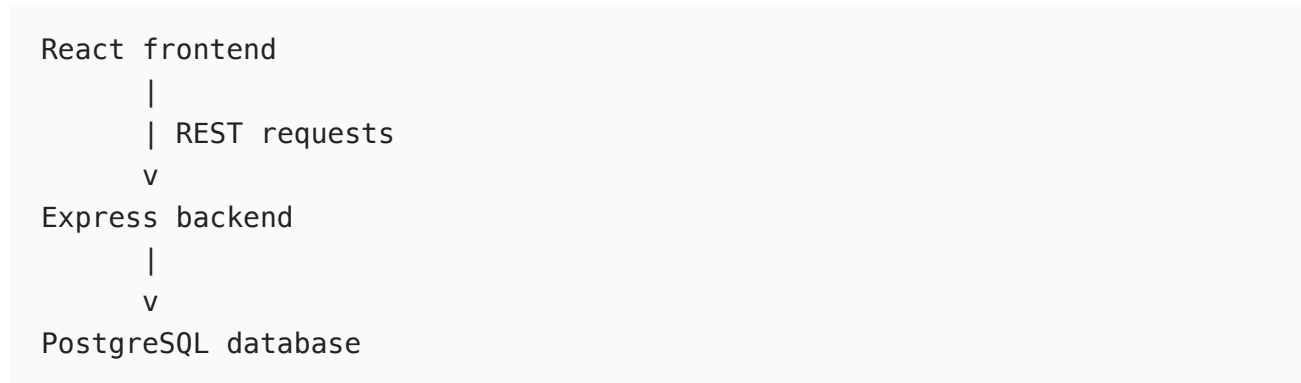
Module: SC2006 Software Engineering

Milestone: Lab 3 — Design and Implementation

Date: 16 September 2026

Recommended architecture

FamPlan should use a simple three-part web architecture:



The Express backend also connects to OneMap, public-data APIs and an email provider. It performs the scheduled data update automatically.

This is a **modular monolith**: one frontend, one backend and one database. Features are separated into folders inside the application, not into independently deployed microservices. This is easier for a student team to build, test and explain.

Core technology stack

Area	Choice	Reason
Language	TypeScript	One language for frontend and backend, with type checking for forms, API data and scoring structures.
Frontend	React with Vite	Supports FamPlan's forms, search results, maps and comparisons with a straightforward development setup.
Backend	Node.js with Express	Provides a small REST API without the complexity of a larger backend framework.
Database	PostgreSQL with Prisma	FamPlan's accounts, preferences, neighbourhoods, facilities and shortlists are relational. Prisma provides migrations and safe database queries.

Area	Choice	Reason
Maps and routes	Leaflet with OneMap	Leaflet displays neighbourhoods and facilities; OneMap supplies Singapore geocoding and supported routes.
Automatic updates	node-cron in the Express backend	Updates configured public datasets daily without an administrator manually starting the process.

Use supported stable releases and commit the package lock file. The team does not need to keep changing versions during the project.

How it supports FamPlan

Frontend

React presents the account forms, preferences, neighbourhood search, profiles, maps, comparisons, shortlist and administration page. Ordinary React state and Context are sufficient; Redux is not required.

Backend

Express handles authentication, validation, search, scoring, shortlist operations and external API calls. The backend can be organised into simple feature folders:

```
account/
preferences/
neighbourhoods/
scoring/
shortlist/
synchronisation/
```

These are modules within one application, not separate services.

Database

PostgreSQL stores accounts, sessions, preferences, neighbourhood and facility data, scoring information, shortlist notes, validated dataset snapshots and synchronisation history.

Important rules should also be enforced in the database, including unique emails, unique shortlist entries, 500-character notes and criterion weights from 0 to 5.

External data

Neighbourhood searches use validated data stored in PostgreSQL instead of calling every public API during each search. The scheduled process downloads, validates and activates a new dataset. If an update fails, FamPlan retains the previous validated dataset.

For the first version, latitude and longitude can be stored directly in PostgreSQL. The supported 500 m, 1 km and 2 km distances can be calculated and tested without introducing PostGIS. PostGIS should only be considered if later performance testing shows a real need.

Essential security

Security can be implemented with a few small supporting libraries rather than another platform:

- Use server-side sessions for login and logout.
- Hash passwords with Argon2id.
- Validate input on both the client and server using Zod.
- Use Prisma or parameterised SQL queries.
- Check record ownership and administrator roles on the server.
- Keep secrets in environment variables.
- Never return or log passwords, reset tokens or session identifiers.
- Display private notes as text rather than untrusted HTML.

This avoids the additional complexity of OAuth, an external identity service or JWT refresh-token handling.

Requirement coverage

Requirement area	Stack support
Accounts and preferences	React forms, Express endpoints, sessions and PostgreSQL
Neighbourhood search	Indexed queries against locally stored validated data
Amenities and maps	Stored facility data, Leaflet and OneMap
Family suitability score	A deterministic TypeScript scoring module
Comparison and shortlist	React views with REST endpoints and relational storage
Automatic synchronisation	<code>node-cron</code> , validation and retained dataset snapshots
Security and privacy	Password hashing, server validation, ownership and role checks

Requirement area	Stack support
Performance and reliability	Local queries, database indexes, pagination and transactions
Accessibility	Semantic HTML, keyboard operation and a text alternative to map markers

The stack makes the NFR targets feasible, but the team must still test them before claiming they are met.

Testing

Use only three testing tools:

- **Vitest** for scoring, distance, validation and data-transformation logic;
- **Supertest** for Express endpoints, authentication and authorisation; and
- **Playwright** for a small number of important browser workflows and accessibility checks.

Performance testing can be added when verifying the 100-concurrent-user NFR; it does not need to complicate the initial skeleton.

Minimal project structure

```
famplan/
├── client/      React frontend
├── server/      Express backend
├── shared/      shared TypeScript types and validation
├── prisma/      database schema and migrations
├── tests/
└── README.md
```

The Express server can serve the compiled React files and REST API, allowing one application deployment connected to one PostgreSQL database.

Implementation order

1. Set up React, Express, TypeScript, PostgreSQL and Prisma.
2. Implement registration, login and family preferences.
3. Import one public dataset and complete neighbourhood search.
4. Add amenities, scoring, comparison, shortlist, maps and routes.
5. Test accessibility, performance and failure recovery.

The team should complete one end-to-end workflow before adding every feature—for example, save one preference through the interface and retrieve it from PostgreSQL.

Technologies not needed

FamPlan should not initially use microservices, Kubernetes, Redis, GraphQL, MongoDB, Elasticsearch, Next.js, NestJS, message queues, machine learning or separate mobile and administrator applications.

These should only be reconsidered if testing identifies a requirement that the proposed stack cannot meet.

Final recommendation

Use React and Vite for the frontend, Node.js and Express for one REST backend, and PostgreSQL with Prisma for persistence. Use Leaflet and OneMap for maps and routes, and run automatic public-data updates within the backend. Keep FamPlan as one modular application and add more technology only when testing demonstrates a genuine need.

This is mostly the same as the Lab 2 recommendation because the updated design model adds detail but does not justify a more complicated architecture.